Practical Project Report

Colored Noise and Hurst Exponent Estimation

Made by Polina Doronicheva

Contents

1 Introduction

The goal of this project is to study colored noise, fractal stochastic processes, and numerical estimation of the Hurst exponent H. The theoretical part reviews fractional Brownian motion (fBm), the fractional Ornstein-Uhlenbeck process (fOU), ARFIMA models, and the main estimation methods for H: R/S analysis, DFA, and the Whittle estimator. The practical part implements generators for processes with a predefined value of H, manual R/S and DFA estimators, a comparison script, and plots of scaling and multifractal-style properties.

The Hurst exponent describes memory and scaling behavior in a time series. The main interpretation is:

$$H < 0.5 \quad \text{anti-persistent behavior,}$$

$$H = 0.5 \quad \text{ordinary Brownian or white-noise reference case,}$$

$$H > 0.5 \quad \text{persistent behavior and long-range dependence.}$$

In the numerical experiment, the tested values are

$$H \in \{0.3,\ 0.5,\ 0.7,\ 0.9\}.$$

This set covers anti-persistent, neutral, persistent, and strongly persistent regimes.

2 Literature Review

2.1 Colored noise and long memory

Colored noise is a random signal whose power spectral density is not constant. A common model is the power law

$$S(f) \sim \frac{1}{f^{\beta}},$$

where $S(f)$ is the power spectral density and β is the spectral exponent. White noise corresponds to $\beta = 0$, pink noise is close to $\beta = 1$, and brown noise is close to $\beta = 2$. In simple words, the color describes which frequencies dominate the signal. White noise looks very irregular because all frequencies are present almost equally, while noise with stronger low-frequency components has longer visible waves and more persistent movement.

In this project, the main colored-noise model is fractional Gaussian noise (fGn). For stationary fGn, the relation

$$\beta = 2H - 1$$

connects the spectral exponent with the Hurst exponent. Therefore, changing H changes both the correlation structure and the spectral color of the noise. This is why fGn is convenient for the practical part: H can be chosen before generation, and then the numerical estimators can be tested against this known value.

The Hurst exponent was introduced in Hurst's hydrological work on long-term reservoir storage [8]. Later, Mandelbrot and Wallis connected the R/S method with long-run statistical dependence [10].

2.2 Fractional Brownian motion and fGn

Fractional Brownian motion was made widely known by Mandelbrot and Van Ness [9]. It is usually denoted by $B_H(t)$, where $H \in (0, 1)$. It is a Gaussian self-similar process with stationary increments. Its covariance function is

$$\mathbb{E}[B_H(t)B_H(s)] = \frac{1}{2}\left(|t|^{2H} + |s|^{2H} - |t-s|^{2H}\right).$$

The increment process of fBm is fractional Gaussian noise:

$$X(t) = B_H(t+1) - B_H(t).$$

This distinction is important for the implementation. fBm is useful for visual sample paths, while fGn is stationary and therefore more convenient for comparing Hurst exponent estimators. Informally, fBm is the accumulated path, while fGn is the sequence of its steps. For estimation, the steps are easier to use because their statistical behavior is stationary.

For simulation, different methods exist: Cholesky decomposition of the covariance matrix, the Hosking method, and Davies-Harte/circulant embedding methods. The practical code uses the Python package `fbm` [5], which provides efficient fBm/fGn generation. The simulation background is related to the Davies-Harte method and stationary Gaussian long-memory processes [3].

2.3 Fractional Ornstein-Uhlenbeck process

The classical Ornstein-Uhlenbeck process is mean-reverting and can be written in the simplified form

$$dX_t = -\theta X_t\, dt + \sigma\, dW_t.$$

The fractional version replaces the Brownian driver with a fractional driver:

$$dX_t = -\theta X_t\, dt + \sigma\, dB_H(t).$$

Fractional Ornstein-Uhlenbeck processes are discussed, for example, by Cheridito, Kawaguchi, and Maejima [2].

In this project, fOU is simulated by a simple Euler-style recursion:

$$X_t = X_{t-1} - \theta X_{t-1}\Delta t + \sigma\, \Delta B_H(t).$$

This implementation is not presented as the most advanced continuous-time fOU simulation method. Its purpose is to clearly show the idea of mean reversion combined with fractional noise. Compared with fBm, fOU should not simply wander away. Even when fractional noise creates persistent movements, the drift term pulls the process back toward a central level.

2.4 ARFIMA models

ARFIMA models generalize ARIMA models by allowing the differencing parameter to be fractional:

$$\phi(B)(1-B)^d X_t = \theta(B)\varepsilon_t,$$

where B is the backshift operator. The fractional differencing operator is expanded as

$$(1-B)^d = 1 - dB + \frac{d(d-1)}{2!}B^2 - \cdots.$$

For simple long-memory cases, the relation

$$H = d + 0.5$$

is often used.

ARFIMA models were introduced in the long-memory time-series literature by Granger and Joyeux [6] and Hosking [7]. In this project, ARFIMA is reviewed but not implemented, because the practical task is focused on fBm/fOU generation and R/S/DFA estimation. The useful point for this report is that ARFIMA gives a discrete-time view of long memory. It describes dependence that decays slowly, similarly to the dependence visible in fGn.

2.5 R/S analysis

R/S analysis is the classical method connected with Hurst's original work [8]. For a window of size s, the local mean is subtracted, cumulative deviations are calculated, the range $R(s)$ is measured, and it is divided by the standard deviation $S(s)$. For scaling processes,

$$\frac{R(s)}{S(s)} \sim s^H.$$

After taking logarithms,

$$\log(R/S) = C + H \log s.$$

Thus, H is estimated as the slope of the log-log regression line. The idea is easy to interpret: if the rescaled range grows quickly when the window size increases, the series has stronger persistence and the estimated H becomes larger.

2.6 DFA

DFA, or detrended fluctuation analysis, was introduced by Peng et al. [12]. The method first builds the cumulative profile

$$Y(k) = \sum_{i=1}^{k} (x_i - \bar{x}),$$

then divides it into windows. In each window, a polynomial trend is fitted and removed. The root-mean-square fluctuation $F(s)$ is then calculated. The scaling law is

$$F(s) \sim s^H.$$

The slope of $\log F(s)$ versus $\log s$ gives the estimate of H. DFA is useful because it first removes a local trend inside every window. Because of this, it is usually less sensitive to simple nonstationary effects than basic R/S analysis.

2.7 Whittle estimation

Whittle estimation is a frequency-domain method. Instead of looking at the series directly in time, it looks at how the variance of the signal is distributed over frequencies. The empirical frequency picture is called the periodogram. The main idea is to choose model parameters so that the theoretical spectral density is close to the observed periodogram.

A simplified Whittle objective can be written as

$$\sum_j \left(\frac{I(\lambda_j)}{f(\lambda_j;\theta)} + \log f(\lambda_j;\theta) \right),$$

where $I(\lambda_j)$ is the periodogram and $f(\lambda_j;\theta)$ is the model spectral density. In simple terms, the estimator tries to fit a curve to the frequency-domain picture of the data.

For long-memory processes, the low frequencies are especially important. If a series has strong persistence, its spectrum usually has more mass near zero frequency. For an ARFIMA-type model, Whittle estimation can be used to estimate the fractional parameter d, and then it can be related to H. The method is important in long-memory statistics and is discussed by Beran [1], Dahlhaus [4], and Robinson [13]. It is reviewed in this report but not implemented in the first version of the code, because implementing it correctly requires choosing and optimizing a spectral model, while the practical requirement here is R/S and DFA.

3 Implementation Logic

I separated the program into small parts: generation, estimation, multifractal-style calculations, plotting, and the experiment script. This made the logic easier to check. I used the package `fbm` for repeated fGn/fBm simulation, because generating many long Gaussian fractional series manually is slow and not the main point of the assignment. For R/S and DFA I wrote the steps directly, because these two estimators are the required practical methods and their algorithms can be explained from the code.

3.1 Generators

The file `generators.py` contains:

- `generate_fgn`: fGn generation with predefined H;
- `generate_fbm`: fBm generation with predefined H;
- `generate_fou`: simple fOU generation;
- `generate_fgn_cholesky`: educational covariance-matrix generator;
- `generate_power_law_noise`: additional $1/f^{\beta}$ colored-noise generator.

The Cholesky generator is included mainly to show the covariance-matrix idea. It is useful for understanding Gaussian process simulation, but it is too slow for the full experiment with many realizations. Therefore, the main experiment uses the faster `fbm` generator.

The key fOU logic is:

```
for t in range(1, n):
    drift = -theta * x[t - 1] * dt
    shock = sigma * (dt**hurst) * increments[t]
    x[t] = x[t - 1] + drift + shock
```

Here, `drift` creates mean reversion and `shock` adds fractional colored noise.

3.2 Manual R/S estimator

The file `estimators.py` implements R/S analysis manually. For each window, the code centers the data, computes cumulative deviations, finds the range, divides by the standard deviation, and then fits a log-log slope.

```
centered = part - np.mean(part)
cumulative = np.cumsum(centered)
r_value = np.max(cumulative) - np.min(cumulative)
s_value = np.std(part, ddof=1)
values.append(r_value / s_value)
```

3.3 Manual DFA estimator

DFA is also implemented manually. The code builds the cumulative profile, splits it into windows, removes a local linear trend, and calculates the root-mean-square fluctuation.

```
profile = np.cumsum(x - np.mean(x))
coeffs = np.polyfit(t, part, deg=order)
trend = np.polyval(coeffs, t)
rms = np.sqrt(np.mean((part - trend) ** 2))
```

The package `nolds` [11] is used only for comparison. It does not replace the manual implementation.

3.4 Multifractal-style fluctuation curves

The file `multifractal.py` computes q-order fluctuation functions $F_q(s)$. This is not a complete multifractal formalism, but it gives useful visual information. Positive values of q emphasize large fluctuations, while negative values emphasize small fluctuations.

4 Numerical Experiment

The experiment is run with:

```
python -m src.main
```

Default parameters are:

- series length: $n = 4096$;
- Hurst values: $0.3, 0.5, 0.7, 0.9$;
- custom estimator repeats: 100 for each H;
- `nolds` estimator repeats: 20 for each H;
- random seed: 42.

The length $n = 4096$ was chosen because it is long enough to provide several window sizes for log-log regression, but still small enough to run the full experiment quickly. It is also a power of two, which is convenient for FFT-based simulation methods. The values $0.3, 0.5, 0.7, 0.9$ cover the main cases: anti-persistence, the neutral Brownian case, moderate persistence, and strong persistence. I used 100 repetitions for the manual estimators to reduce random variation in the average result. The `nolds` functions are slower, so 20 repetitions are used only as a reference comparison. The seed 42 has no special mathematical meaning; it is used for reproducibility, so the same figures and tables can be regenerated.

The estimators compared are:

- `custom_rs`;
- `custom_dfa`;
- `nolds_rs`;
- `nolds_dfa`.

The main error metric is

$$\text{error} = |\widehat{H} - H|.$$

This metric was chosen because the true H is known in the simulation. So the simplest accuracy check is the distance between the average estimate $\widehat{H}$ and the value used in the generator. It is not a special estimator from one paper; it is a direct absolute error for numerical comparison. The standard deviation is also reported because two methods can have similar average error but different stability.

5 Results

5.1 Generated sample paths

Figure 1 shows fGn sample paths. For small H, the process is more anti-persistent. For large H, longer persistent movements become more visible.

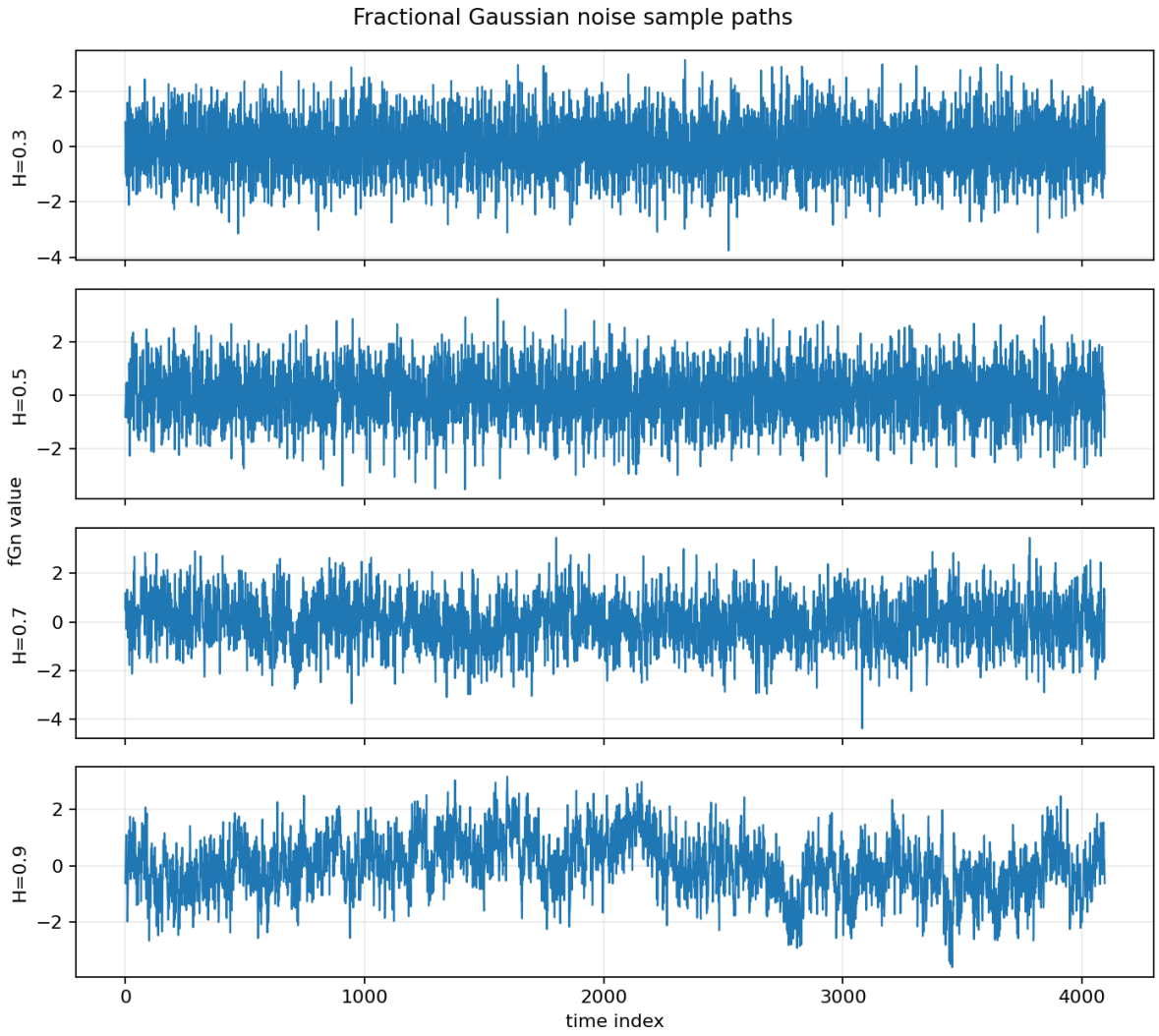

Figure 1: Fractional Gaussian noise sample paths for different H.

In the plotted realizations, the $H = 0.3$ and $H = 0.5$ series change direction more often, while the $H = 0.7$ and especially $H = 0.9$ series have longer runs of movement in the same direction. This agrees with the expected transition from anti-persistence to persistence.

Figure 2 shows fBm sample paths. The paths become visually smoother as H increases.

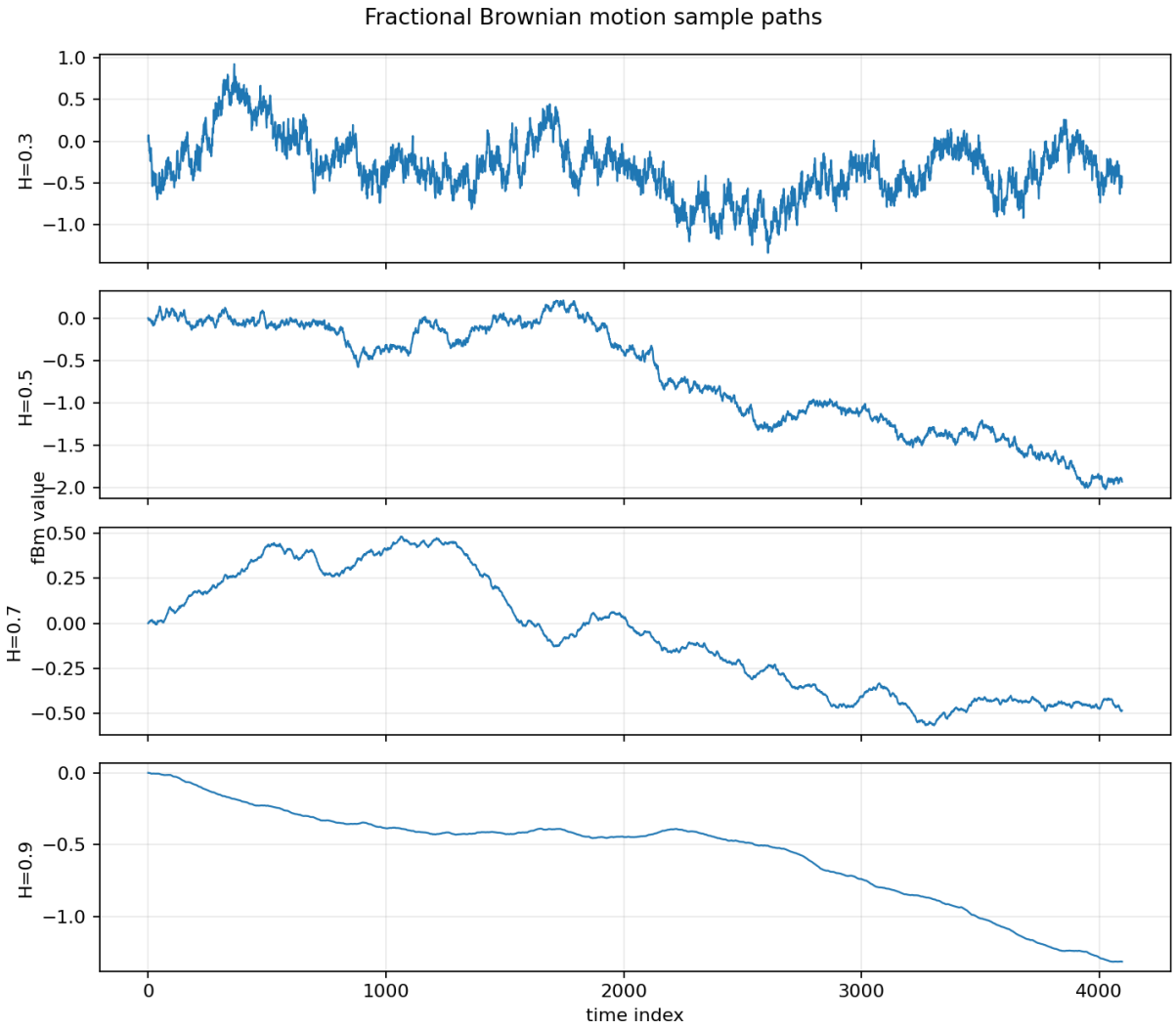

Figure 2: Fractional Brownian motion sample paths for different H.

The same effect is visible for fBm, but now in accumulated form. The path with smaller H is more jagged, while for larger H the curve becomes smoother and long trends are easier to see.

Figure 3 shows fOU sample paths. They fluctuate around a central level, which demonstrates mean reversion.

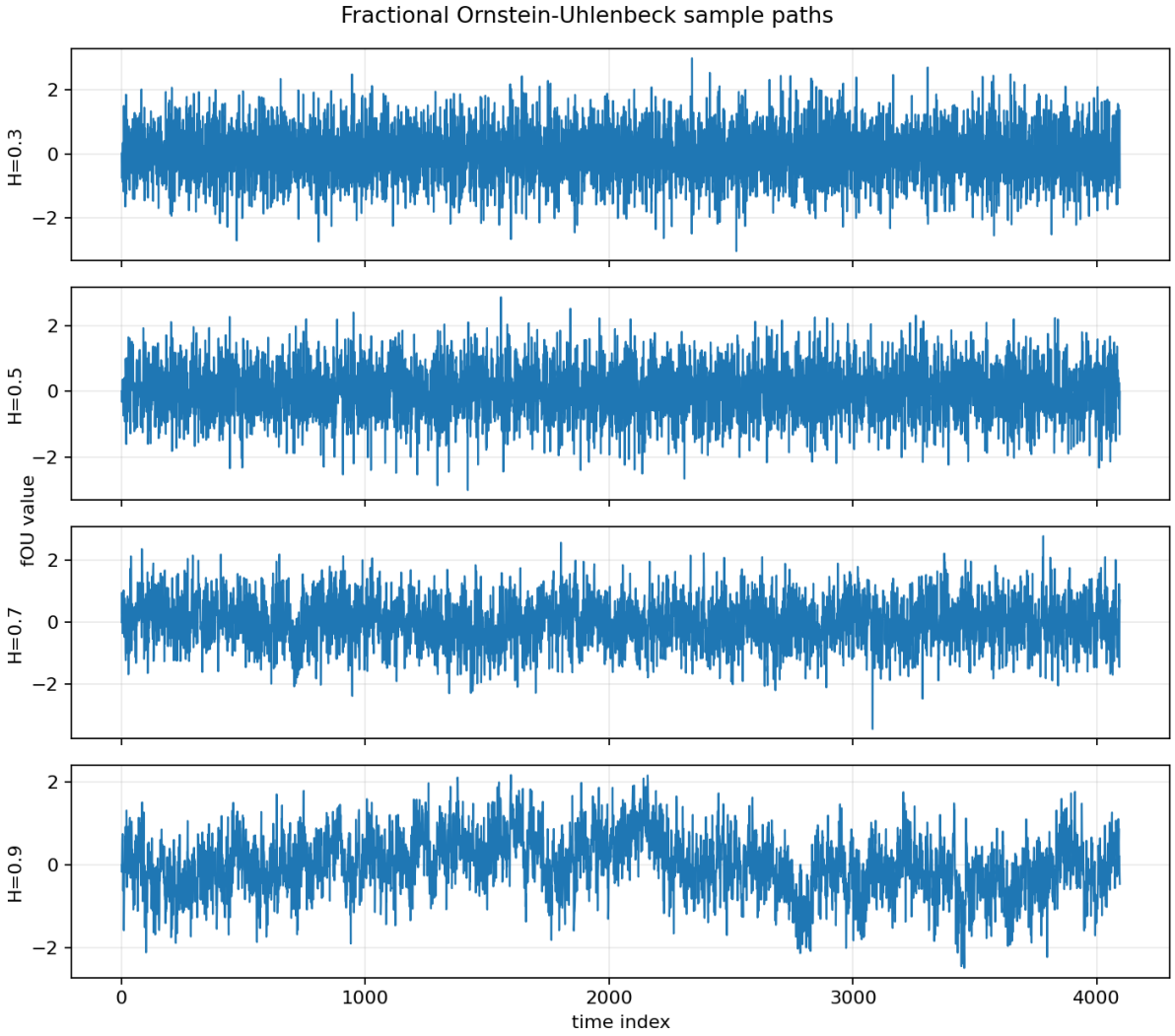

Figure 3: Fractional Ornstein-Uhlenbeck sample paths.

All fOU paths stay around a central level, so the mean-reverting part of the model is visible. At higher H, deviations are more persistent, but the process still returns toward the center.

5.2 Scaling plots

Figure 4 shows the R/S log-log plot for fGn with $H = 0.7$. The slope is used as the R/S estimate of the Hurst exponent.

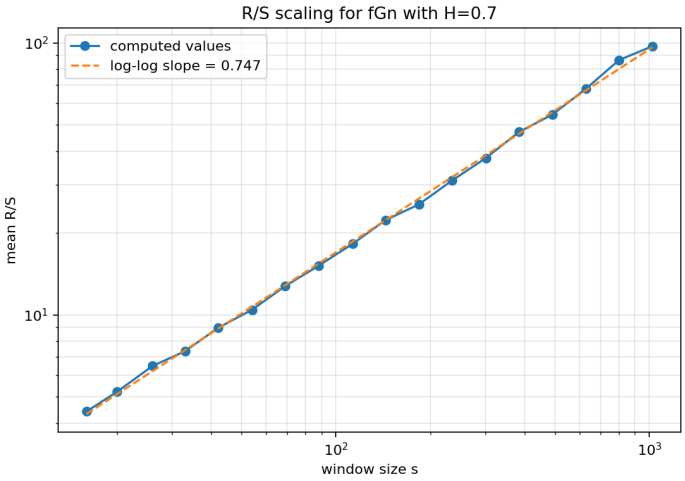

Figure 4: R/S scaling plot for fGn with $H = 0.7$.

The points are close to a straight line in log-log coordinates. In this example, the fitted slope is about 0.747, which is higher than the true value $H = 0.7$. This shows the finite-sample positive bias that R/S can have in one realization.

Figure 5 shows the DFA log-log plot for the same process.

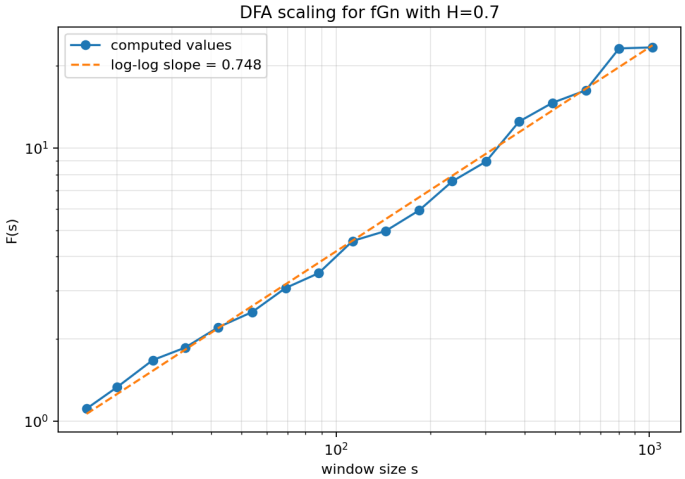

Figure 5: DFA scaling plot for fGn with $H = 0.7$.

DFA also gives an almost linear relation in log-log coordinates. The slope is about 0.748, close to the R/S slope for this single example, but the repeated experiment below shows that DFA is more accurate on average.

5.3 Accuracy comparison

Table 1 gives the mean estimates and absolute errors.

Table 1: Comparison of Hurst exponent estimates.

H	Method	Mean	Std	Count	Error
0.3	custom_dfa	0.2987	0.0225	100	0.0013
0.3	custom_rs	0.3839	0.0178	100	0.0839
0.3	nolds_dfa	0.3257	0.0108	20	0.0257
0.3	nolds_rs	0.3498	0.0134	20	0.0498
0.5	custom_dfa	0.4963	0.0311	100	0.0037
0.5	custom_rs	0.5492	0.0226	100	0.0492
0.5	nolds_dfa	0.5169	0.0174	20	0.0169
0.5	nolds_rs	0.5047	0.0186	20	0.0047
0.7	custom_dfa	0.6963	0.0390	100	0.0037
0.7	custom_rs	0.7141	0.0286	100	0.0141
0.7	nolds_dfa	0.7128	0.0161	20	0.0128
0.7	nolds_rs	0.6516	0.0246	20	0.0484
0.9	custom_dfa	0.8888	0.0454	100	0.0112
0.9	custom_rs	0.8503	0.0300	100	0.0497
0.9	nolds_dfa	0.9125	0.0227	20	0.0125
0.9	nolds_rs	0.7810	0.0277	20	0.1190

Figure 6 compares the estimated values with the ideal diagonal line.

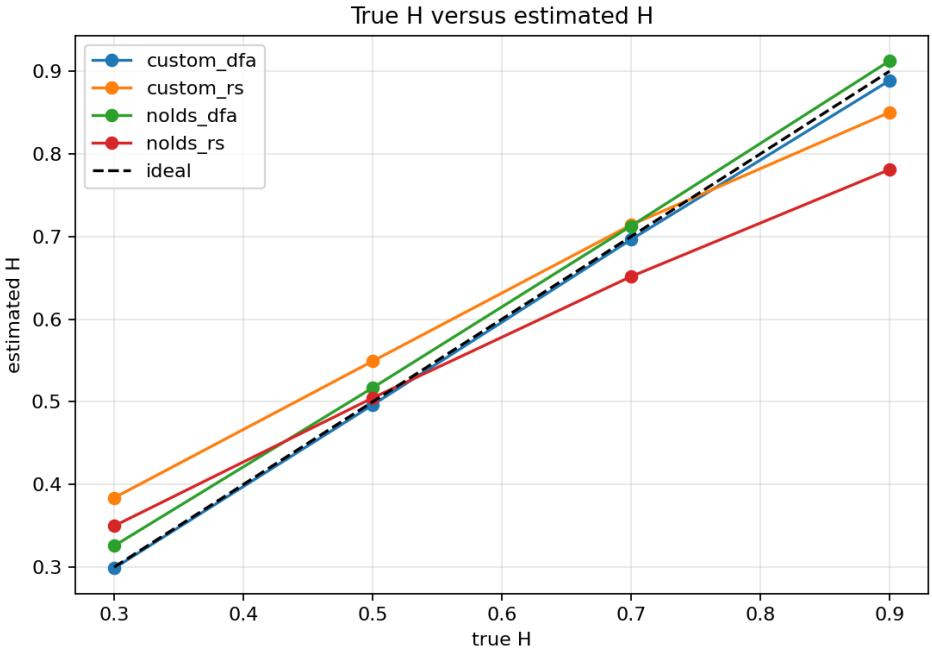

Figure 6: True H versus estimated H.

The curves generally follow the ideal diagonal, so all methods detect the correct ordering of H. The custom DFA line is closest to the ideal line, while `nolds_rs` visibly underestimates the largest value $H = 0.9$.

Figure 7 shows the absolute errors. In this experiment, the manual DFA implementation gives the smallest errors for most values of H. R/S preserves the correct order of H, but it has stronger finite-sample bias.

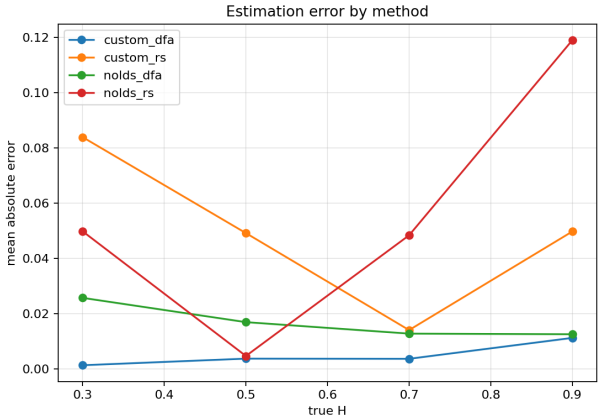

Figure 7: Absolute estimation error by method.

This plot makes the numerical comparison clearer. Custom DFA has the smallest error for most tested values, R/S has larger finite-sample errors, and the largest visible error appears for `nolds_rs` at $H = 0.9$.

5.4 Multifractal-style properties

Figure 8 shows q-order fluctuation curves. Different values of q emphasize different fluctuation sizes. These curves are included as a visual multifractal-style diagnostic.

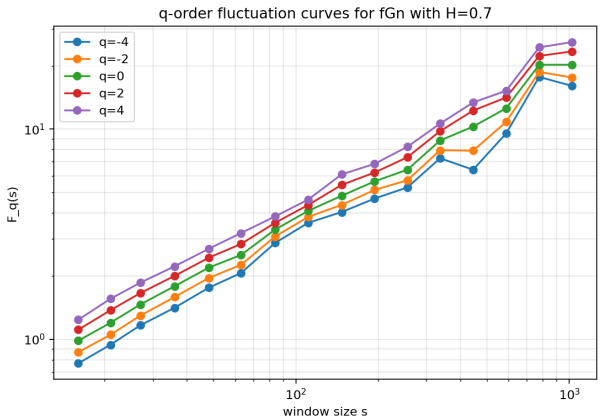

Figure 8: q-order fluctuation curves $F_q(s)$ for fGn with $H = 0.7$.

The $F_q(s)$ curves increase with scale, which confirms that fluctuations grow on larger windows. The separation between different q values means that small and large fluctuations are weighted differently.

6 Discussion

The numerical results are consistent with the theory. The estimates increase as the true value of H increases. The lag-1 autocorrelation is negative for $H = 0.3$, close to zero for $H = 0.5$, and positive for $H = 0.7$ and $H = 0.9$. This confirms that the generated fGn realizations have the expected anti-persistent, neutral, and persistent behavior.

The manual DFA estimator performs best in the default experiment. This is reasonable because DFA removes local trends and often behaves more stably than basic R/S analysis. The R/S estimator is simpler and historically important, but it can be biased in finite samples.

The comparison with `nolds` is included as an additional check. However, the logic of the required estimators is written manually. The code follows the algorithmic descriptions from the literature and does not simply call library functions for the main result.

7 Conclusion

The project includes both a literature review and a practical numerical part. The reviewed models are fBm, fOU, and ARFIMA. The reviewed estimation methods are R/S analysis, DFA, and Whittle estimation. The implemented code generates fGn, fBm, and fOU with predefined H, estimates H using manual R/S and DFA, compares the estimates with library methods, and builds scaling and multifractal-style plots.

The experiment shows that DFA gives the most accurate results in this setup, while R/S still correctly reflects the ordering of the Hurst exponent. The generated figures also demonstrate the expected visual differences between anti-persistent, neutral, and persistent processes.

References

[1] Jan Beran. *Statistics for Long-Memory Processes*. Chapman and Hall, 1994.

[2] Patrick Cheridito, Hideyuki Kawaguchi, and Makoto Maejima. Fractional ornstein-uhlenbeck processes. *Electronic Journal of Probability*, 8:1–14, 2003.

[3] Peter F. Craigmile. Simulating a class of stationary gaussian processes using the davies-harte algorithm, with application to long memory processes. *Journal of Time Series Analysis*, 24(5):505–511, 2003.

[4] Rainer Dahlhaus. Efficient parameter estimation for self-similar processes. *The Annals of Statistics*, 17(4):1749–1766, 1989.

[5] fbm package. fbm: Fractional brownian motion and fractional gaussian noise in python. https://pypi.org/project/fbm/, 2026. Accessed 2026-06-17.

[6] C. W. J. Granger and Roselyne Joyeux. An introduction to long-memory time series models and fractional differencing. *Journal of Time Series Analysis*, 1(1):15–29, 1980.

[7] J. R. M. Hosking. Fractional differencing. *Biometrika*, 68(1):165–176, 1981.

[8] H. E. Hurst. Long-term storage capacity of reservoirs. *Transactions of the American Society of Civil Engineers*, 116:770–799, 1951.

[9] Benoit B. Mandelbrot and John W. Van Ness. Fractional brownian motions, fractional noises and applications. *SIAM Review*, 10(4):422–437, 1968.

[10] Benoit B. Mandelbrot and James R. Wallis. Robustness of the rescaled range r/s in the measurement of noncyclic long run statistical dependence. *Water Resources Research*, 5(5):967–988, 1969.

[11] nolds package. nolds: Nonlinear measures for dynamical systems. https://pypi.org/project/nolds/, 2026. Accessed 2026-06-17.

[12] C.-K. Peng, S. V. Buldyrev, S. Havlin, M. Simons, H. E. Stanley, and A. L. Goldberger. Mosaic organization of dna nucleotides. *Physical Review E*, 49:1685–1689, 1994.

[13] P. M. Robinson. Gaussian semiparametric estimation of long range dependence. *The Annals of Statistics*, 23(5):1630–1661, 1995.

A Code Fragments

A.1 fGn and fBm generation

```
def generate_fgn(n: int, hurst: float, length: float = 1.0,
                 seed: int | None = None) -> np.ndarray:
    if seed is not None:
        np.random.seed(seed)
    model = FBM(n=n, hurst=hurst, length=length,
                method="daviesharte")
    values = np.asarray(model.fgn(), dtype=float)
    return standardize(values)

def generate_fbm(n: int, hurst: float, length: float = 1.0,
                 seed: int | None = None) -> np.ndarray:
    if seed is not None:
        np.random.seed(seed)
    model = FBM(n=n, hurst=hurst, length=length,
                method="daviesharte")
    values = np.asarray(model.fbm(), dtype=float)
    return values
```

A.2 fOU generation

```
def generate_fou(n: int, hurst: float, theta: float = 1.2,
                 sigma: float = 0.8, dt: float = 1.0,
                 x0: float = 0.0,
                 seed: int | None = None) -> np.ndarray:
    increments = generate_fgn(n=n, hurst=hurst, seed=seed)
    x = np.zeros(n, dtype=float)
    x[0] = x0
    for t in range(1, n):
        drift = -theta * x[t - 1] * dt
        shock = sigma * (dt**hurst) * increments[t]
        x[t] = x[t - 1] + drift + shock
    return x
```

A.3 R/S analysis

```
def estimate_hurst_rs(series, scales=None):
    x = np.asarray(series, dtype=float)
    if scales is None:
        scales = make_scales(len(x))
    rs_values = []
    used_scales = []
    for scale in scales:
        blocks = len(x) // scale
        if blocks < 2:
            continue
        values = []
        for block in range(blocks):
```

```
            part = x[block * scale : (block + 1) * scale]
            centered = part - np.mean(part)
            cumulative = np.cumsum(centered)
            r_value = np.max(cumulative) - np.min(cumulative)
            s_value = np.std(part, ddof=1)
            if s_value > 0:
                values.append(r_value / s_value)
        if values:
            used_scales.append(scale)
            rs_values.append(np.mean(values))
    slope, _, _, _, _ = linregress(np.log(used_scales),
                                   np.log(rs_values))
    return float(slope), used_scales, rs_values
```

A.4 DFA

```
def estimate_hurst_dfa(series, scales=None, order=1):
    x = np.asarray(series, dtype=float)
    if scales is None:
        scales = make_scales(len(x))
    profile = np.cumsum(x - np.mean(x))
    flucts = []
    used_scales = []
    for scale in scales:
        blocks = len(profile) // scale
        if blocks < 2:
            continue
        rms_values = []
        t = np.arange(scale)
        for block in range(blocks):
            part = profile[block * scale : (block + 1) * scale]
            coeffs = np.polyfit(t, part, deg=order)
            trend = np.polyval(coeffs, t)
            rms = np.sqrt(np.mean((part - trend) ** 2))
            if rms > 0:
                rms_values.append(rms)
        if rms_values:
            used_scales.append(scale)
            flucts.append(np.sqrt(np.mean(np.asarray(rms_values) ** 2)))
    slope, _, _, _, _ = linregress(np.log(used_scales),
                                   np.log(flucts))
    return float(slope), used_scales, flucts
```

A.5 Multifractal-style fluctuations

```
if q == 0:
    fq = np.exp(0.5 * np.mean(np.log(variances)))
else:
    fq = (np.mean(variances ** (q / 2.0))) ** (1.0 / q)
```